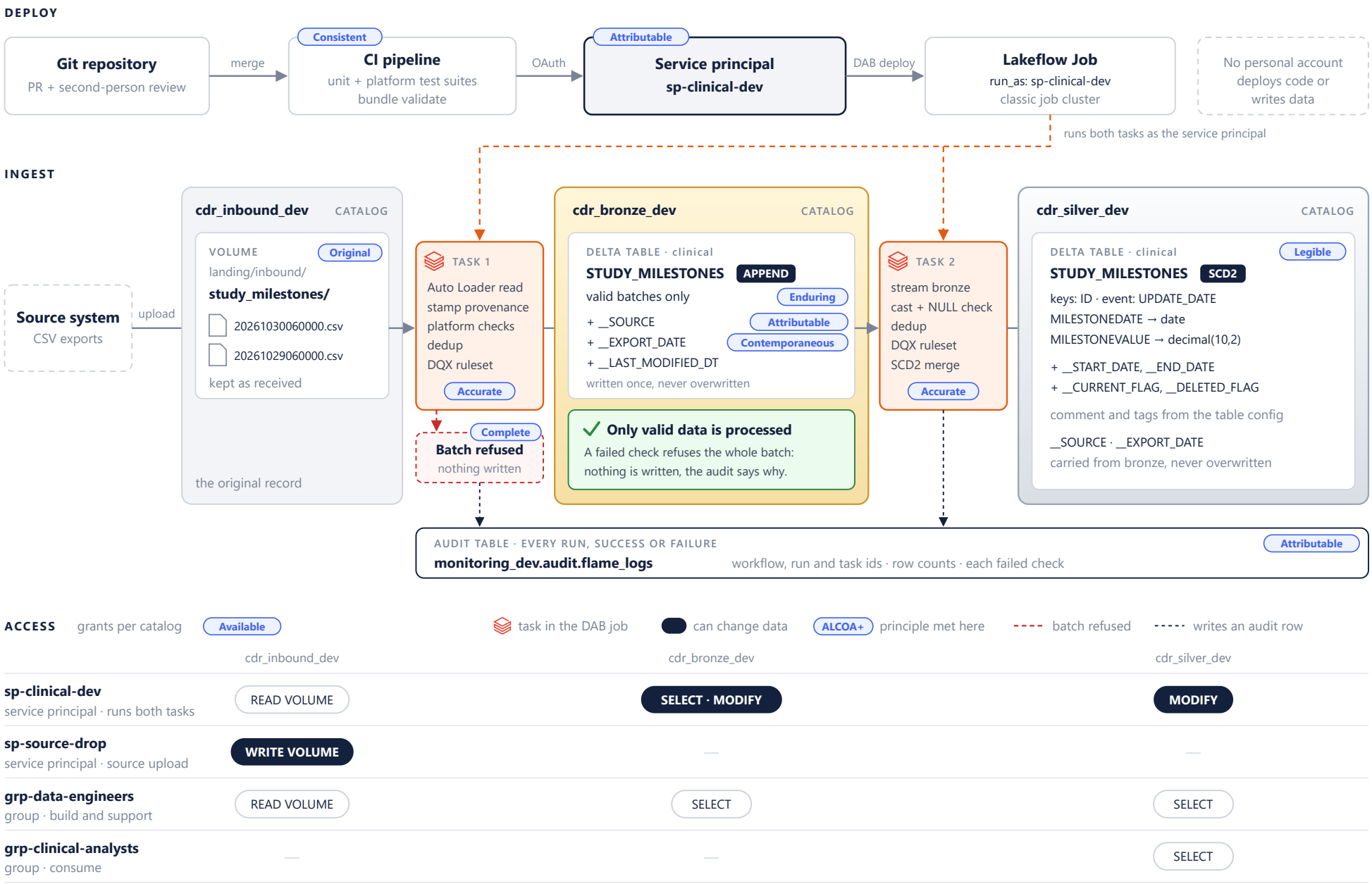
dbx-flame
v0.2.2

dbx-flame on GxP environments

Meeting ALCOA+ with Configuration Driven Pipelines



Configuration, deployed with DABs

Each hop is one task in a Declarative Automation Bundle. Its `named_parameters` are the dbx-flame settings, validated at Start before any data is read. CI deploys the bundle, and `env: ${bundle.target}` resolves every catalog to `{catalog}_{env}`.

resources/clinical_data_repository.yml

```yaml
# Inbound → Bronze
- task_key: study_milestones_bronze
  job_cluster_key: *cluster
  python_wheel_task:
    package_name: dbx-flame
    entry_point: dbx-flame
    named_parameters:
      <<: *identity   # job, run and task ids for the audit row
      catalog: cdr_bronze
      env: ${bundle.target}
      metadata_path: /Volumes/cdr_bronze_${bundle.target}/clinical/metadata/
      source.origin: csv
      source.path: /Volumes/cdr_inbound_${bundle.target}/landing/inbound/
      source.directory: study_milestones
      policies.checks_file: ${var.quality_checks}/study_milestones_bronze.yml
      output.verb: append
      output.schema_name: clinical
      output.table: STUDY_MILESTONES

# Bronze → Silver
- task_key: study_milestones_silver
  depends_on: [{task_key: study_milestones_bronze}]
  job_cluster_key: *cluster
  python_wheel_task:
    package_name: dbx-flame
    entry_point: dbx-flame
    named_parameters:
      <<: *identity
      catalog: cdr_silver
      env: ${bundle.target}
      metadata_path: /Volumes/cdr_silver_${bundle.target}/clinical/metadata/
      table_config: ${workspace.file_path}/genie_metadata/study_milestones.yml
      source.origin: table
      source.catalog: cdr_bronze
      source.schema_name: clinical
      source.table: STUDY_MILESTONES
      policies.checks_file: ${var.quality_checks}/study_milestones_silver.yml
      output.verb: scd2
      output.schema_name: clinical
      output.table: STUDY_MILESTONES
      output.keys: ID
      output.event_time.column: UPDATE_DATE
```

genie_metadata/study_milestones.yml

```yaml
comment: >-
  SCD2 silver table.
  Filter on __CURRENT_FLAG = 'Y' to get the latest state of each record.
  This table has Multi-study data, one study may have multiple milestones.
tags:
  data_class: gxp
  owner: clinical_data
  source_system: CDR
columns:
  - name: ID
    comment: Study milestone unique key.
    tags:
      filter_key: "true"
  - name: STUDY_ID
    comment: Filter by this column to get data from a single study.
    tags:
      filter_key: "true"
  - name: MILESTONEDATE
    cast:
      target_type: date
      format: yyyy-MM-dd
  - name: MILESTONEVALUE
    cast:
      target_type: decimal(10,2)
```

quality_checks/study_milestones_silver.yml

```yaml
# DQX ruleset: error refuses the batch, warn is logged
- criticality: error
  check:
    function: is_not_null
    arguments:
      column: STUDY_ID
- criticality: warn
  check:
    function: is_not_null
    arguments:
      column: MILESTONEDATE
```

How dbx-flame meets ALCOA+

Attributable
Every task runs as the service principal set in the bundle, never as a person. Every run writes an audit row with its workflow, run and task ids, and `__SOURCE` records the file or table each row was read from.

Legible
Silver holds typed columns in Delta. The table config sets the table description and each column's comment and tags in Unity Catalog, so people and Genie can read what the data means.

Contemporaneous
`__EXPORT_DATE` comes from the 14-digit stamp in the file name, the time the source produced the export. A file without a stamp is refused. `__LAST_MODIFIED_DT` records when the job wrote the row.

Original
Files stay in the inbound volume as received. `__SOURCE` and `__EXPORT_DATE` are written once, where the data enters, and never overwritten on promotion.

Accurate
Each batch passes the framework checks (missing stamp, a cast that would silently produce NULL, unparseable dates, unexpected new columns) and your DQX ruleset before it is written.

Complete
A refused batch writes nothing, so no table ever holds part of a batch. The audit row names each failed check and its row count. SCD2 keeps every version, and deletes are flagged, never removed.

Consistent
CI deploys the same reviewed bundle to every target, so dev and prod run identical task definitions. Invalid settings are rejected at Start, before any data is read.

Enduring
Bronze is append-only and silver keeps full history in Delta, in catalogs that only the service principal can change.

Available
Groups read through grants on whole catalogs, so reviewers and inspectors can get the data, its history and the audit trail when they ask.

Runs on classic job clusters. Serverless rejects the SCD2 cache, the UPSERT autoMerge conf and the `foreachBatch` context.

Databricks · Delta Lake · DQX · Unity Catalog